# Lab # 05

**COURSE CODE: SE-206L**

COURSE TITLE:

# Software Construction & Development

LAB TITLE

## Designing Specifications: Preconditions Postconditions

LAB ASSESSMENT TABLE:

| Obtained Marks | Total Marks |
|---|---|
| | |


Submitted To: Engr. Rizwan Shah.

Submitted By: Muhammad Danial.

Registration – No: 24ABSWE0021.

Semester: 5th

Date: 16-09-2026.

Dept. of Software Engineering

UET Peshawar, Abbottabad Campus.


Instructor Sign : _____

# Lab 5:

# Designing Specifications: Preconditions Postconditions

**Repository Link:** https://github.com/danial-uet/Software-Construction-Labs./tree/master/Lab5/SpecificationLab

## Objective

This lab introduces method specifications, focusing on preconditions, postconditions, and behavioral equivalence. Students will learn how to write formal contracts (specifications) for methods using JavaDoc, handle precondition violations by "failing fast," and evaluate the strength and clarity of specifications.

## Learning Outcomes

- After completing this lab, students will be able to:
- Understand and write method specifications using requires (preconditions) and (postconditions). effects
- Determine if two different method implementations are behaviorally equivalent from a client's perspective.
- Apply the "fail fast" principle when a client violates a precondition.
- Differentiate between declarative and operational specifications.
- Compare the strength of specifications (weaker preconditions and stronger postconditions).

## Manual Lab Task

### Explanation:

• The JavaDoc comments map the theoretical requires and effects clauses to @param, @return , and @throws.

• The specification clearly states that `val` must occur exactly once.

If the client passes an array where val occurs zero times or multiple times, the precondition is violated. Instead of returning a random index or -1 , the method explicitly throws an IllegalArgumentException to "fail fast" at the site of the bug.

## Solution

For manual lab task, single Java Class was created by me inside SpecsAndContracts Package in SpecificationLab Project:

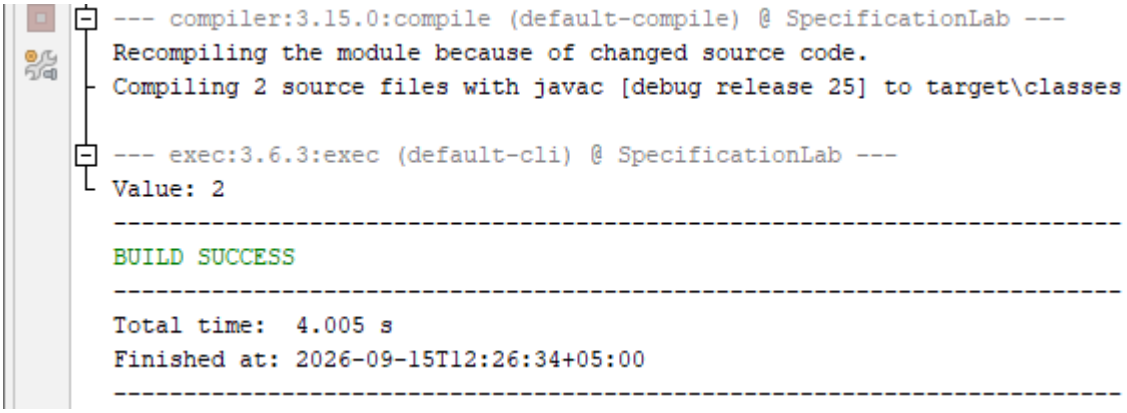
● SpecificationLab.java:

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/main/java/com/university/lab/specificationlab/SpecificationLab.java

● ArraySearcher.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/main/java/SpecsAndContracts/ArraySearcher.java

## Output:




## Lab Task 1 — Behavioral Equivalence

### Objective:

Understand how the client's perspective and the specification dictate if two implementations are equivalent.

### Implementation ( SearchStrategies.java ):

• Write a method findFirst(int[] arr, int val) that loops from index 0 to arr.length- 1 and returns the first matching index, or arr.length if not found.

• Write a method findLast(int[] arr, int val) that loops backwards from arr.length - 1 to 0 and returns the matching index, or -1 if not found.

### Test ideas

• Write a JUnit test demonstrating a case where the two methods return different results (e.g., when the array contains duplicates or is missing the value).

• Write a JUnit test demonstrating a case where both methods return the same result (when the element appears exactly once).

### Solution

For 1st lab task, single Java Class along with its Junit testing class was created by me inside Lab5Tasks Package in SpecificationLab Project:

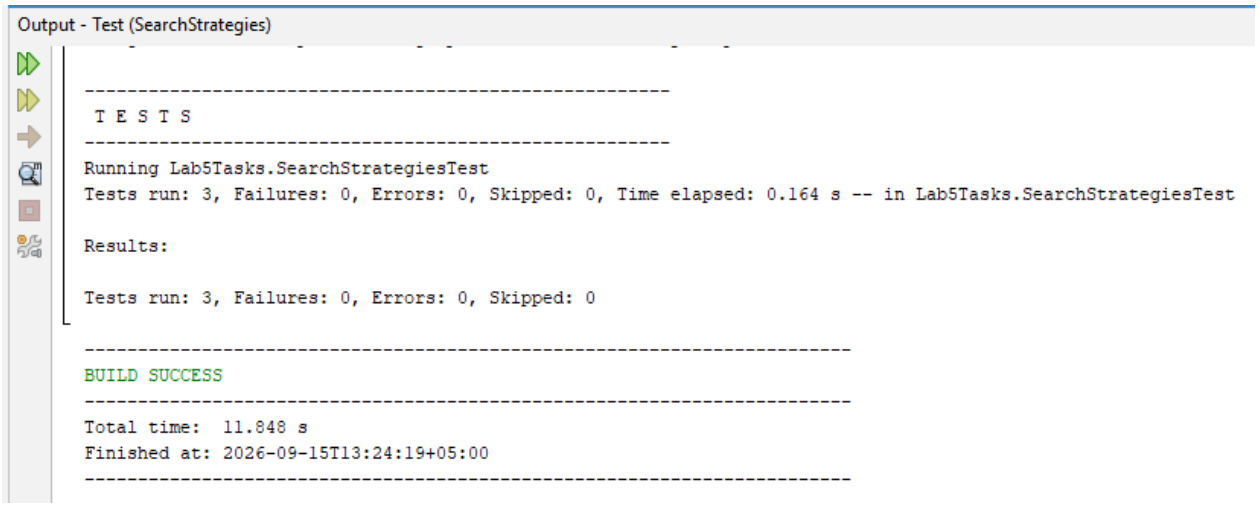
● SearchStrategies.java, (Java Class):

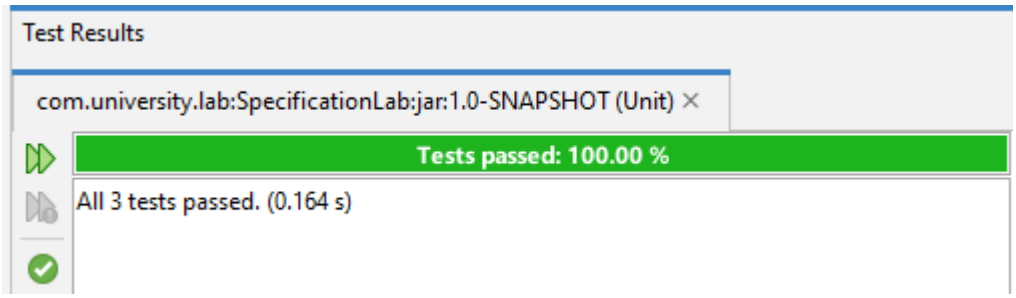
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/main/java/Lab5Tasks/SearchStrategies.java

● SearchStrategiesTest.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/test/java/Lab5Tasks/SearchStrategiesTest.java

## Output:




## Lab Task 2 — Failing Fast on Precondition Violations

### Objective:

Practice throwing unchecked exceptions when clients fail to meet contractual obligations.

### Implementation ( MathUtils.java ):

• Create a method public static double calculateGravitationalPotentialEnergy(double altitude)

• Add a JavaDoc comment stating: @param altitude in meters relative to sea level. Requires altitude >= 0 .

• In the method body, write code that throws an IllegalArgumentException if altitude < 0 (failing fast).

### Expected Output:

• Write a JUnit test utilizing assertThrows(IllegalArgumentException.class, () -> ...) to ensure your method fails fast when given a negative altitude.

## Solution

For 2nd lab task, single Java Class along with its Junit testing class was created by me inside Lab5Tasks Package in SpecificationLab Project:

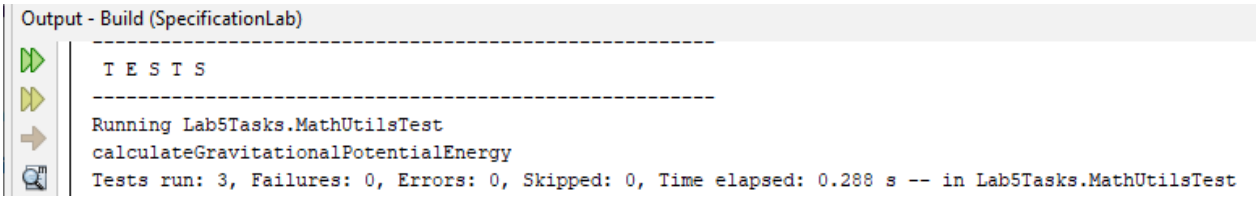
● MathUtils.java, (Java Class):

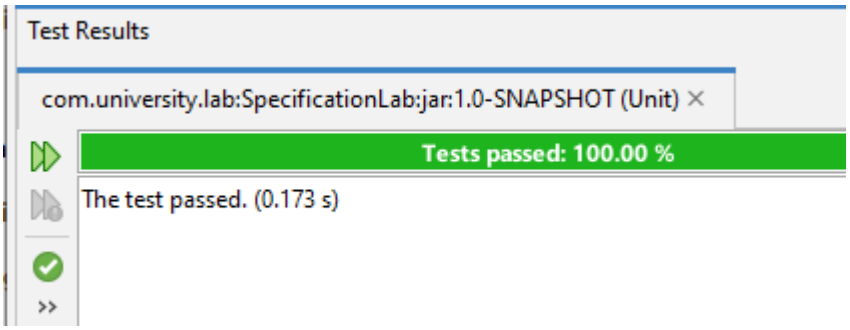
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/main/java/Lab5Tasks/MathUtils.java

● MathUtilsTest.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/test/java/Lab5Tasks/MathUtilsTest.java

## Output:




## Lab Task 3 — Mutation Contracts

### Objective:

Understand that mutation is disallowed unless explicitly stated in the specification.

### Implementation ( ListFormatter.java ):

○ Write a method public static void sortInPlace(List lst) that explicitly sorts the list passed into it (mutating method).

○ Write a method public static List toLowerCase(List lst) that returns a new list where all strings are lowercase, leaving the original list untouched.

### Test ideas

• Assert that the list passed to toLowerCase remains unchanged after the method is called to verify no implicit mutation occurred.

## Solution

For 3rd lab task, single Java Class along with its Junit testing class was created by me inside Lab5Tasks Package in SpecificationLab Project:

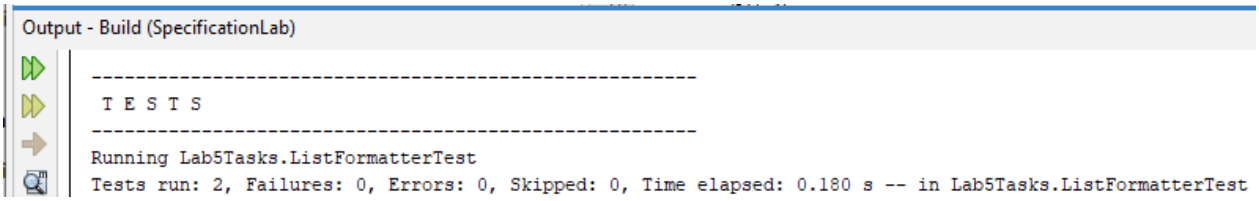
● ListFormatter.java, (Java Class):

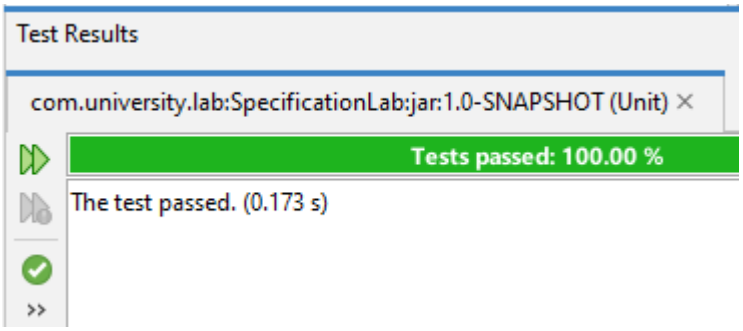
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/test/java/Lab5Tasks/ListFormatterTest.java

● ListFormatterTest.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/test/java/Lab5Tasks/ListFormatterTest.java

## Output:




## Lab Task 4 — Immutability for Safer Contracts

### Objective:

Understand how returning mutable objects complicates contracts and reduces changeability.

### Tasks:

● Implement a class Authenticator .

● Write an insecure method public char[] getMitId(String username) that returns a 9-digit ID as a character array.

● Write a test where the client receives the username) that char[] , modifies the first 5 digits to '*' , and unintentionally corrupts a cached version stored inside Authenticator .

● Write a secure method public String getMitIdSecure(String that returns an immutable username) String . Explain in your README why this immutable return type acts as a safer firewall.

## Solution

For 4th lab task, single Java Class along with its Junit testing class was created by me inside Lab5Tasks Package in SpecificationLab Project:

● Authenticator.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/main/java/Lab5Tasks/Authenticator.java

● AuthenticatorTest.java, (Junit test class):

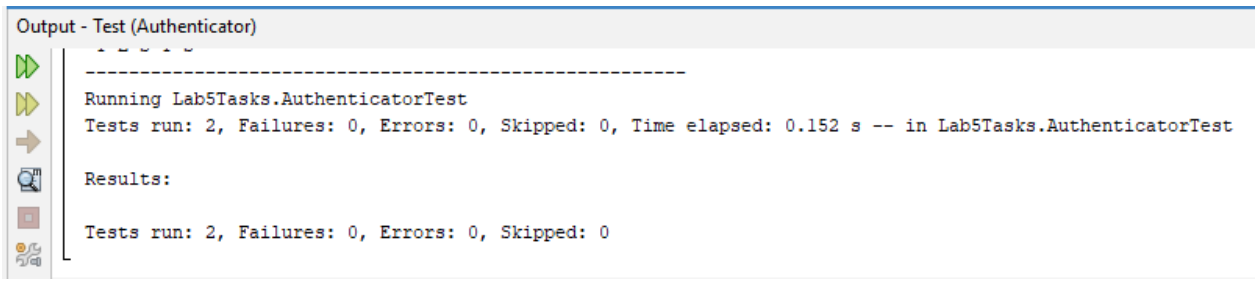
https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/test/java/Lab5Tasks/AuthenticatorTest.java

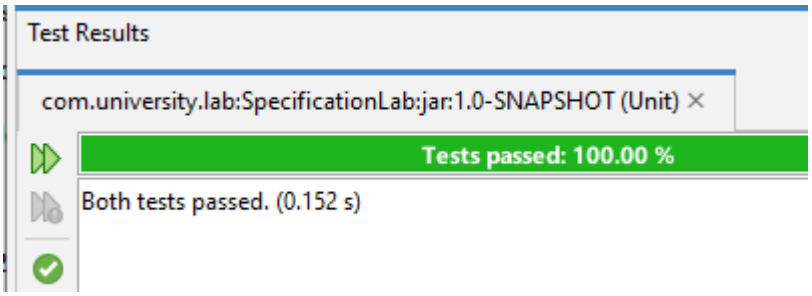
**README Explanation Section**

**Why Immutable Return Types Act as a Safer Firewall:**

Returning mutable objects (such as char[]) exposes internal class references directly to external callers, allowing untrusted code to modify internal state without going through explicit methods. Returning an immutable type like String creates a structural firewall: callers cannot mutate the object after creation, guaranteeing that internal caches remain secure, accurate, and protected against unintentional corruption.

## Output:




## Lab Task 5 — Declarative vs. Operational Specs

### Objective:

Refactor operational specifications into clear, declarative specifications.

## Tasks:

● Assume you have a method joinStrings(List<String>. words, String delimiter)

● Write a bad, operational JavaDoc for this method that describes the internal for loop, the if-statements used to avoid trailing delimiters, and the exact string concatenation steps.

● Write a good, declarative JavaDoc for this method that only focuses on the final output (e.g., "Returns the concatenation of elements in order, with the delimiter inserted between each adjacent pair").

● Submit both JavaDocs as comments in your joinStrings source code file.

## Solution

For 5th lab task, single Java Class along with its Junit testing class was created by me inside Lab5Tasks Package in SpecificationLab Project:

● StringUtil.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/main/java/Lab5Tasks/StringUtil.java

● StringUtilTest.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab5/SpecificationLab/src/test/java/Lab5Tasks/StringUtilTest.java

## Output:

